

P4-Contiguous Memory Allocation

Based on "Contiguous Memory Allocation", OSC 10th edition, chapter 9. page P-48 **BUT** slightly different, so follow the instructions below.

There are no starter files for this project.

Contiguous Memory Allocation

Section 9.2 (<https://jigsaw.vitalsource.com/books/9781119320913/epub/OPS/c09.xhtml#c09-sec-0002>) of the OSC book described several algorithms for contiguous memory allocation: **First-Fit**, **Best-Fit** and **Worst-Fit**.

This project asks you to manage allocations within a memory pool of size MEMSIZE. Your program support six different requests:

1. **Allocate** N bytes for a process using one of the 3 allocation algorithms
2. **Free** all allocations for a given process
3. **Show** the status of the memory pool – allocated and free blocks
4. **Read** a script – a sequence of commands from a file, and execute them
5. **Compact** the allocations, making them into one contiguous block. (This somewhat resembles the operation of a mark-sweep garbage collector in C#)
6. **Exit**

MEMSIZE has a value of 80. So we can think of the memory pool as holding 80 bytes. (If you prefer, you can think of it as holding 80 KBytes, where allocations are made in multiples of 1 KByte)

Processes are named as a single letter, A through Z.

Here is an example of the Show command after some allocations and frees:

```
AAAAAAAAA.....BBBBBBBBBFFFFFFFFFFFFFFFFGGG..CCCCCH.....DDDDD.....
```

(MEMSIZE was chosen as 80 so the display would fit nicely onto our console screen)

The **6** command formats are as follows:

A <name> <size> <algo>	Allocate <size> bytes for process <name> using algorithm <algo>. <algo> can be any of F for First-Fit, B for Best-Fit or W for Worst-Fit. Eg: A P 20 F Allocate 20 bytes for process P using First-Fit Eg: A X 14 B Allocate 14 bytes for process X using Best-Fit
F <name>	Free all the allocations owned by <name> Eg: F P Free all allocations owned by process P
S	Show the state of the memory pool
R <file>	Read the script in the file called <file> and execute each command. Eg: R MEMO.TXT
C	Compact the memory pool, <i>sliding</i> all allocations to lower addresses so they become one contiguous block, and so that all the free space lies to the right as one contiguous block.
E	Exit

Here is an example script:

```
A  A  10  F
A  X  10  F
A  B  10  F
A  X  20  F
A  C   5  F
A  X  15  F
A  D   5  F
F  X
E
```

The first Show command produces:

```
AAAAAAAAAAXXXXXXXXXXBBBBBBBBBBXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXDDDDDD.....
```

After the “F X” command, it becomes:

```
AAAAAAAAA.....BBBBBBBBBB.....CCCC.....DDDD.....
```

Compaction

Here is an example pool before compaction:

```
AAAAAAAAA.....BBBBBBBBBBFFFFFFFFFFFFFFFFGGG..CCCCCH.....DDDD.....
```

And after compaction:

```
AAAAAAAAABBBBBBBBBBFFFFFFFFFFFFFFFFGGGCCCHDDDD.....
```

The OSC book suggests the “Compact” command. In reality, however, an Operating System cannot move memory allocations around, willy-nilly. What if that memory included pointers, such as a linked-list. What would go wrong? But virtual machines, as used by Java and C#, *do* track pointers. They can, and do, compact memory.

Grading

Here is the script we will run to grade submissions:

```
AA 10 F
AX 10 F
AB 10 F
AX 20 F
AC 5 F
AX 15 F
AD 5 F
FX
S
AE 25 F
AF 15 F
AG 3 B
AH 1 W
S
C
E
```

What to Submit

Your program should accept input from the standard input and read a file if "R" is given.

Upload **memory.zip** with all the project files including:

1. output.txt - showing your program compile and run
2. memory.c
3. memory.h (if necessary)
4. any other .c and .h files as necessary

Grading Rubric

- 5 pt/each for First-Fit, Best-Fit, Worst-Fit
- 3 pt Free
- 4 pt Compaction
- 3 pt Read file